Department of Artificial Intelligence & Data Science,
K. K. Wagh Institute of Engineering Education and Research Nashik

A
Mini Project Report
On

"REAL-TIME OBJECT DETECTION USING DEEP LEARNING"

SUBMITTED BY

Shaikh Mohammed Ateeb (B231311115)
Tejal Nitin Pawar (B231311105)

Course Code: 2301403 Course Name: Deep Learning Lab
Class: Final Year Div: A

GUIDED BY

Prof. Ifat J. Shaikh

SUBMITTED TO

Department of Artificial Intelligence & Data Science
K. K. Wagh Institute of Engineering Education and Research,
Nashik
Academic Year 2026-2027

INDEX

Chapter No	Contents	Page No.
1.	Abstract	3
2.	Introduction	4
3.	Scope	5
4.	Requirement Analysis	6
5.	System / Model Architecture	7
6.	Software Requirements	8
7.	GUI	9
8.	Source Code	10
9.	Testing	11
10.	Conclusion	12

CONTRIBUTION STATEMENT

Both team members, Shaikh Mohammed Ateeb (B231311115) and Tejal Nitin Pawar (B231311105), contributed equally to this project. Work was shared jointly across all areas, including requirement analysis, front-end interface design, model integration and inference pipeline development, testing, and preparation of this report. No part of the project was the sole responsibility of a single member.

CHAPTER 1
ABSTRACT

This project focuses on the development of a browser-based application for real-time object detection that uses a pretrained deep learning model to identify and localise everyday objects directly from a live webcam feed. Most AI-powered vision applications rely on cloud servers and paid APIs; this project demonstrates that accurate, real-time inference can instead run entirely inside the browser with no backend at all. Built using HTML5, CSS3 and JavaScript for the interface, and ONNX Runtime Web for inference, the application loads a pretrained YOLOv8s model (trained by Ultralytics on the Open Images V7 dataset, covering 600 object classes) in ONNX format and runs it on the GPU through WebGPU, falling back to WebAssembly on the CPU when unavailable. Each webcam frame is resized and letterboxed to 640 x 640 pixels and passed through the network in a single forward pass, producing 8,400 candidate object boxes with class scores. Duplicate boxes are removed using non-maximum suppression, and detections are drawn live on a canvas overlay with their class name and confidence percentage. By incorporating concepts such as convolutional neural network architecture, model export and conversion pipelines, image preprocessing, and client-side inference optimisation, the project offers a practical understanding of deploying deep learning models in production without training infrastructure, servers, or recurring API costs.

CHAPTER 2
INTRODUCTION

Object detection is a core computer vision task that goes beyond simple image classification by locating and naming multiple objects within a single frame. Traditionally, deploying such models required a backend server, a GPU-equipped cloud instance, and often a paid vision API, all of which add cost, latency, and privacy concerns, since video frames must be uploaded to a third party for processing.

This project addresses that gap by running the entire detection pipeline inside the user's own browser. A convolutional neural network, pretrained on a large general-purpose dataset, is converted to the ONNX format and executed locally using ONNX Runtime Web, so no video frame ever leaves the device.

The chosen model, YOLOv8s trained on Open Images V7, recognises 600 everyday object categories, including bottle, mobile phone, watch, glasses, mug, laptop and person, making it general-purpose enough for a wide range of real-world use. Comparable earlier approaches commonly rely on MobileNet-based image classification (1000 labels, but only one label for the whole image) or smaller detection models such as COCO-SSD or EfficientDet (80 classes only); this project instead uses a true multi-object detector trained on hundreds of categories, giving noticeably better results on a busy, real-world webcam feed.

Beyond the deep learning aspect, this project gave us hands-on experience with the full lifecycle of deploying a trained model: exporting PyTorch weights to ONNX, optimising a neural network for in-browser inference, handling asynchronous model loading and WebRTC camera access, and building a responsive, real-time user interface around the model's output.

CHAPTER 3
SCOPE

The application has been designed to be lightweight, fully client-side, and general-purpose, so it can recognise a wide variety of everyday objects without any project-specific training. The primary objectives covered within the scope of this project include:

- Providing a fully client-side, real-time object detection experience directly from a webcam feed.
- Using a single pretrained CNN capable of recognising hundreds of general object categories, without any custom training.
- Converting and optimising the trained model for in-browser execution using ONNX Runtime Web, with GPU acceleration via WebGPU and automatic CPU fallback.
- Decoding raw model output into bounding boxes, applying confidence filtering and non-maximum suppression, and rendering results live with class name and confidence.
- Offering usability features: camera selection for multi-camera devices, an option to hide people/clothing detections, and detection smoothing to avoid flicker.
- Laying groundwork for future fine-tuning (already prototyped on Google Colab) to improve accuracy further without re-architecting the application.

Key Functional Modules:

1. Model Loading Module – fetches and initialises the ONNX model and class labels on page load, with a loading indicator.
2. Camera Module – requests webcam access via WebRTC, lists available cameras, and streams the live feed to a <video> element.
3. Inference Pipeline – preprocesses frames (letterbox resize to 640x640, normalisation), runs the ONNX model, decodes the 605 x 8,400 output into labelled boxes, filters by confidence (≥35%), and removes duplicate boxes via non-maximum suppression (IoU ≥45%).
4. Detection Rendering Module – draws bounding boxes and labels on a canvas overlay in real time and updates a side panel with the most confident detection and a list of other detected objects.

The application goes beyond simple image classification. It is a real-time, fully offline-capable web application developed with the vision of showing that practical, general-purpose deep learning inference does not require any server-side infrastructure.

CHAPTER 4
REQUIREMENT ANALYSIS

4.1 Functional Requirements

5. Load the pretrained ONNX model and class labels on page load, with visible loading status.
6. Request webcam access and handle permission-denied, no-camera, and camera-busy errors gracefully.
7. List and allow switching between available camera devices, remembering the user's choice.
8. Run inference continuously on live video frames and decode detections above a confidence threshold.
9. Remove duplicate/overlapping detections of the same object using non-maximum suppression.
10. Optionally filter out person/clothing-related classes via a toggle.
11. Keep a detected object visible for a short duration after it was last seen, to reduce flicker.
12. Display the most confident detection and a list of other currently visible objects with confidence percentages.
13. Allow starting and stopping the camera/detection loop on demand.

4.2 Non-Functional Requirements

14. Fully client-side: no video frame or detection result is ever sent to a server.
15. Works offline after the first load, since the model and inference library are bundled locally.
16. Responsive layout across desktop and mobile screen sizes.
17. Real-time performance: GPU acceleration through WebGPU where available, with automatic WebAssembly/CPU fallback.
18. Must run on a localhost or HTTPS origin only, since browsers restrict camera access otherwise.
19. Clear, accessible status and error messaging for model loading and camera failures.

CHAPTER 5
SYSTEM / MODEL ARCHITECTURE

5.1 Model Architecture Overview

The application uses YOLOv8s ("You Only Look Once", version 8, small size), a convolutional neural network pretrained by Ultralytics on Open Images V7 (about 1.7 million photos spanning 600 object categories). No training is performed in this project; the pretrained weights are used as-is and only converted to the ONNX format for the browser to run.

- Backbone: convolution layers that turn the image into features – edges, then textures, then object parts.
- Neck: combines features extracted at different scales, so both small and large objects can be found.
- Head: for every candidate box, outputs the box position and a score for each of the 601 class slots.
- Input: a 640 x 640 colour image with pixel values normalised between 0 and 1.
- Output: a table of 605 x 8,400 numbers – 4 box values plus 601 class scores for each of the 8,400 candidate boxes.

5.2 Detection Pipeline (Data Flow)

Webcam frame → letterbox resize to 640x640 → pixel normalisation → ONNX Runtime Web forward pass → per-box highest-scoring class → confidence threshold (≥35%) → coordinate rescale to the original frame → non-maximum suppression (IoU ≥45% within the same class) → optional person/clothing filter → 1-second detection persistence → bounding boxes and labels drawn on the canvas overlay.

5.3 Design Rationale

A single-pass, multi-class detector was chosen over an image classification model such as MobileNet, which only produces one label for the entire image and performs poorly on busy, multi-object webcam scenes. It was also chosen over smaller 80-class detectors such as COCO-SSD or EfficientDet, which cannot recognise everyday items like watches, glasses or pens. Running inference fully client-side, with no server or database layer, keeps all video data private to the user's device and removes any dependency on backend infrastructure, in line with the project's local-first approach to deploying deep learning models.

CHAPTER 6
SOFTWARE REQUIREMENTS

6.1 Front End

- Technologies: HTML5, CSS3, JavaScript (vanilla, no framework)
- Rendering: HTML5 <video> element for the camera feed, Canvas 2D API for the detection overlay

6.2 Model & Inference

- Model: YOLOv8s trained on Open Images V7 (Ultralytics), exported to ONNX (opset 17)
- Runtime: ONNX Runtime Web (ort.webgpu.min.js) – GPU via WebGPU, with WebAssembly/CPU fallback
- Browser APIs: WebRTC getUserMedia for camera access

6.3 Model Preparation (one-time, offline)

- Language: Python 3
- Libraries: Ultralytics, onnx, onnxslim (export_model.py converts yolov8s-oiv7.pt to ONNX + labels.json; not required to run the app itself)

6.4 Deployment

- No backend server and no database are used; any static file server works.
- Development: served locally via “python -m http.server 8080” (or VS Code Live Server) on localhost, since browsers only allow camera access on https:// or http://localhost.

CHAPTER 7
GRAPHICAL USER INTERFACE

Technology Used: HTML5, CSS3, vanilla JavaScript, with the Canvas 2D API for overlay rendering.

Functionality: Live webcam view with real-time bounding boxes and labels, start/stop camera controls, a camera device selector, and a hide-people toggle.

User Interface: A status banner (model loading / ready / error), the video feed with a transparent canvas overlay, a controls row, a camera dropdown, a toggle checkbox, and a prediction panel showing the top label, confidence percentage and bar, plus a list of other currently visible objects.

Design Aesthetic: Clean, card-based layout with a responsive grid, accessible status messaging (aria-live region), and a visible confidence bar for quick feedback.

User-Friendly: Works across desktop and mobile, with clear error states (camera permission denied, no camera found, model load failure) so users always know what is happening.

CHAPTER 8
SOURCE CODE

Key project files:

- index.html – page structure: video element, overlay canvas, controls, prediction panel
- style.css – layout, responsive design, status/prediction styling
- script.js – model loading, webcam handling, frame preprocessing, inference, output decoding, duplicate removal, detection smoothing, and canvas drawing
- export_model.py – one-time script converting the pretrained PyTorch weights to the ONNX + labels.json files used by the app
- models/ – yolov8s-oiv7.onnx (pretrained weights) and labels.json (601 class names)
- lib/ – local copy of ONNX Runtime Web, so the app works fully offline

```
const MODEL_URL = "models/yolov8s-oiv7.onnx";
const LABELS_URL = "models/labels.json";
const INPUT_SIZE = 640;
const MIN_SCORE = 0.35;

async function loadModel() {
  labels = await (await fetch(LABELS_URL)).json();
  // "webgpu" runs the network on the graphics card; falls back to "wasm" (CPU).
  session = await ort.InferenceSession.create(MODEL_URL, {
    executionProviders: ["webgpu", "wasm"]
  });
}
```

Figure 8.1: Model loading logic from script.js

```
const ORDERED_MODEL_URL = "models/yolov8s-oiv7.onnx";
// Candidate boxes are decoded, filtered by MIN_SCORE, then deduplicated
// with nonMaximumSuppression() before being drawn on the canvas overlay.
```

Figure 8.2: Detection decoding settings from script.js

Source Code: github link https://github.com/Mohammed-Ateeb/real-time-object-detection

CHAPTER 9
TESTING

The application underwent functional, integration and informal accuracy testing:

9.1 Unit / Component Testing

- Verified the frame preprocessing step correctly letterboxes frames of different aspect ratios to 640x640 without distortion.
- Verified the decoding step correctly extracts the highest-scoring class per candidate box and discards boxes below the 35% confidence threshold.
- Verified non-maximum suppression removes overlapping boxes of the same class above 45% IoU while keeping separate objects intact.

9.2 Integration Testing

- Simulated the full workflow: model load → start camera → live detection → stop camera, across multiple camera devices.
- Verified error-handling paths: camera permission denied, no camera available, model load failure.

9.3 Accuracy Testing

The model was run inside the app on 42 photos from Wikimedia Commons. A photo counts as correct when the expected class is found with at least 35% confidence (the app's threshold).

Object	Correct
Bottle	7 / 12
Mobile phone	4 / 12
Mug / coffee cup	5 / 6
Watch	8 / 12

This is a small, informal test, not a formal benchmark. Several of the phone and bottle photos show the object very small, in a crowd, or not fully visible, so those rows understate the model's real performance.

9.4 Cross-Browser / Performance Testing

- Chrome and Edge (WebGPU): faster inference and smoother, more frequent box updates.
- Other browsers (WebAssembly/CPU fallback): noticeably slower, with a several-times lower detection rate.

CHAPTER 10
CONCLUSION

This project demonstrates that a pretrained convolutional neural network can deliver real-time, multi-class object detection entirely inside a web browser, without training a custom model, without a backend server, and without any paid API. By combining YOLOv8s pretrained on Open Images V7 with ONNX Runtime Web, the application recognises 600 everyday object categories directly from a live webcam feed while keeping all video data on the user's own device.

Beyond the end result, the project gave us practical, end-to-end experience with the deep learning deployment pipeline: understanding a CNN's backbone-neck-head architecture, exporting trained weights to a portable format, optimising a model for client-side inference, and building the preprocessing and post-processing logic (letterboxing, confidence filtering, non-maximum suppression) that turns raw model output into a usable interface.

Current limitations include lower confidence scores due to the large number of classes, sensitivity to lighting and object size, and slower performance on devices without WebGPU support. We have also prototyped a fine-tuning pipeline (using Google Colab's free GPU) to further improve accuracy on a balanced subset of Open Images V7, which we plan to apply as future work, along with displaying live FPS, counting detected objects, and exploring the medium YOLOv8 model for higher accuracy on stronger hardware.